Sudoku Solver: Report

Zeynep Eskicioğlu ; Valentina Macek

June 1, 2026

1 Introduction

This project investigates the use of machine learning techniques for solving Sudoku puzzles. While classical approaches such as backtracking, constraint propagation, and evolutionary algorithms are well-established and highly effective, we wanted to explore how well different machine learning methods can perform on this task, both in terms of accuracy and runtime.

The idea of applying machine learning to Sudoku came from the work presented in [1], which motivated us to explore different strategies in which ML models could be integrated as helpers in solving Sudoku. In particular, we implemented three strategies: full-grid prediction, step-by-step next-cell prediction, and ML-guided backtracking.

2 Dataset

We used a 9 Million Sudoku Puzzles and Solutions from Kaggle, where each sample is a pair of 81-character strings; one for the puzzle (zeros represent empty cells) and one for the solution. From this dataset, we randomly sampled 50.000 puzzles. The distribution of empty cells in this sample is shown in Figure 1.

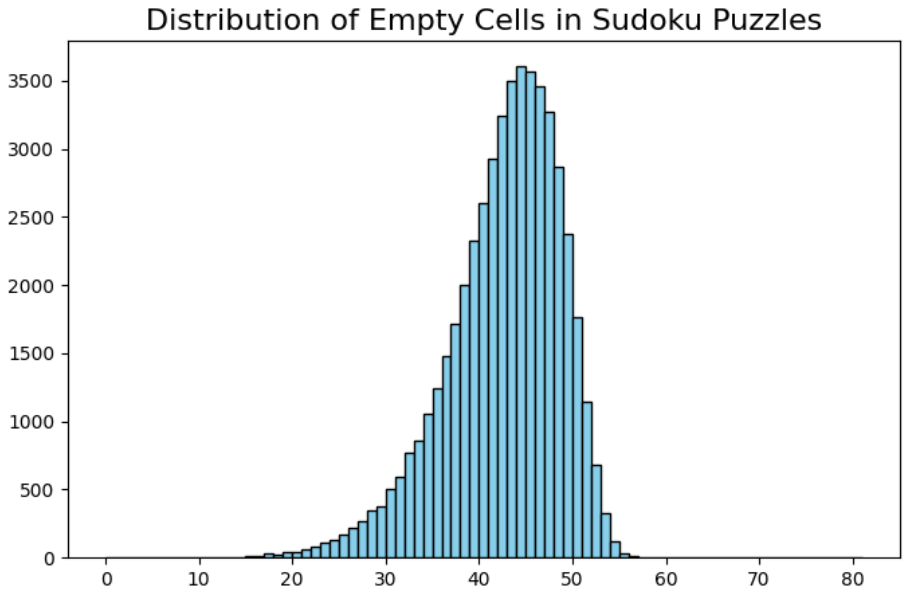


Figure 1: Distribution of empty cells in our sample

We then constructed a training set of 40.000 puzzles, a validation set of 5.000 puzzles, and a test set of 5.000 puzzles.

3 Data preprocessing

For each puzzle, we first convert 81-character string into a 9×9 integer grid and apply one-hot encoding: each cell value is represented as a 9-dimensional binary vector. This produces a tensor of shape $9 \times 9 \times 9$, where each of the nine channels corresponds to a digit from 1 to 9. For all models except CNN, we then flatten this tensor into a 1-dimensional vector of length 729. The corresponding solution grid is converted into a vector of 81 class labels (0–8), one for each cell. For the convolutional model, we keep the one-hot encoded grid in its original 3-dimensional form, preserving spatial structure so that convolutional filters can operate across rows, columns, and sub-grids. In this case, solution labels are reshaped back into a 9×9 grid.

4 Methodology

4.1 Evaluation metrics

We evaluate model performance using two accuracy measures: *per-cell accuracy* and *per-puzzle accuracy*. Per-cell accuracy computes the proportion of individual cells predicted correctly across all puzzles, while per-puzzle accuracy counts

a puzzle as correct only if all 81 cells match the ground truth. Reporting both metrics allows us to distinguish models that make locally accurate predictions from those capable of producing complete, valid Sudoku solutions.

4.2 Strategy 1: Full-grid prediction

The model takes the full one-hot encoded puzzle as input and predicts all 81 cells at once. For Random Forest and XGBoost we trained 81 independent classifiers, one per cell. For MLP and CNN, a single model outputs predictions for all cells jointly.

This serves as our baseline. The main weakness is that predictions are made without enforcing Sudoku constraints, so the output grid can be locally accurate but globally invalid.

4.3 Strategy 2: Step-by-step next-cell prediction

Instead of predicting everything at once, the model fills cells one at a time. At each step it picks the empty cell it is most confident about (highest softmax probability), fills it in, and re-encodes the updated grid for the next iteration. We applied this to the MLP and CNN.

4.4 Strategy 3: ML-guided backtracking

Here we combine ML with classical backtracking. Whenever the solver needs to fill an empty cell, it asks the ML model for a probability distribution over digits 1–9 and tries them in that order rather than always starting from 1. If a choice leads to a contradiction, standard backtracking takes over.

This keeps the correctness guarantee of backtracking while letting the model guide the search toward more promising branches. We used this with RF, XGBoost, MLP and CNN.

5 Results

Figure 2 provides a comparison of per-cell and per-puzzle accuracy across all models and strategies. The results reveal a strong mismatch between local and global performance for Strategy 1 (full-grid prediction) and Strategy 2 (step-by-step next-cell prediction): although these models achieve moderate per-cell accuracy, none of them are able to solve complete puzzles, resulting in 0% per-puzzle accuracy. This is true even for the best performing model built with Strategy 1, Xgboost. This indicates that small local errors accumulate rapidly and prevent the model from reaching a valid global Sudoku solution. In contrast, for the ML-backtracking strategy, per-cell and per-puzzle accuracy coincide because any successfully completed puzzle must be a fully valid Sudoku grid; once the solver reaches a valid solution, every cell is necessarily correct.

Moreover, we can see that strategy 2 of solving only the most confident cell each iteration doesn't improve the model per-cell accuracy from strategy 1. This

means that even if the model is confident about filling a particular empty cell, that choice is not necessarily the best move for the puzzle as a whole.

Comparing accuracy across different models reveals a clear trend. For models with already high per-cell accuracy, backtracking compensates for the remaining local mistakes, allowing the solver to recover and still reach a valid final grid. As a result, the MLP, CNN, and XGB models paired with backtracking achieve perfect per-puzzle accuracy, clearly outperforming all other strategies. This demonstrates that ML models alone are not yet reliable enough to solve Sudoku end-to-end, but they become highly effective when guiding a search-based solver.

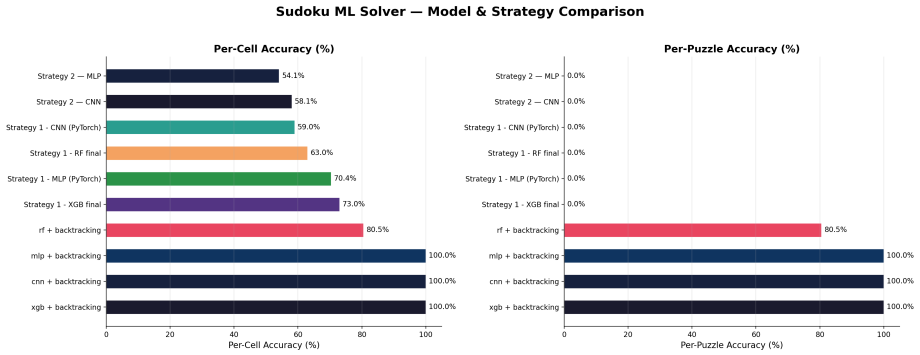


Figure 2: Accuracy comparison across all models.

To further evaluate this effectiveness, we compared ML-guided backtracking with pure backtracking. Tables 1 and 2 summarize our findings.

Table 1 reports runtime and success comparisons between pure backtracking and ML-guided backtracking on 50 random puzzles from the test set. Pure backtracking solves all puzzles extremely quickly, with average runtimes around 0,02 s. ML-guided backtracking also solves all puzzles but is consistently slower due to the overhead of running the ML model at each decision step.

Table 1: Backtracking comparison on random puzzles.

Method	**Success**	**Avg (s)**
Pure BT	50/50	0.0190
ML-BT	50/50	0.1340

To investigate whether puzzles with more empty cells might give the ML-guided solver an advantage, we also compared both methods on 30 harder puzzles from the test set. Although these puzzles are more challenging for humans, they are not computationally difficult for classical backtracking algorithm, again completing all puzzles extremely quickly. ML-guided backtracking remains slower for the same reason as before: the inference cost of the ML model outweighs any reduction in search depth. It seemed that this was not a representative set of truly hard puzzles, and it remains possible that on genuinely difficult instances

the ML-guided approach could help backtracking by providing a more effective ranking of candidate digits.

Table 2: Backtracking comparison on hard puzzles from the test set.

Method	**Success**	**Avg (s)**
Pure BT	30/30	0.0129
ML-BT	30/30	0.1154

6 Conclusion

Taken together, these results highlight the complementary strengths of machine learning and classical search. ML models provide useful local guidance but fail to solve puzzles independently, while backtracking guarantees correctness but lacks prioritization. When combined, the two approaches produce a solver that is both accurate and robust. However, our runtime experiments show that pure backtracking remains faster than the ML-guided variant on puzzles with an average of 45 empty cells, since the ML inference overhead outweighs any reduction in search depth. Unfortunately, the puzzles in our dataset were not sufficiently challenging for classical backtracking, preventing us from assessing whether ML guidance could accelerate search on genuinely difficult instances.

References

[1] Sunanda Jana, Mrinmoy Sen, Abhinandan Khan, Arnab Kumar Maji, and Rajat Kumar Pal. Analysing the effectiveness of different machine learning approaches towards solving sudoku puzzle. In *Proceedings of International Conference on Data, Electronics and Computing*, pages 25–41, Singapore, 2025. Springer Nature Singapore.